

Part A: Short Questions (2–3 marks each)

1. Why are software quality requirements considered non-functional requirements?

Functional requirements define *what* a system must do (e.g., process a payment). Quality requirements are non-functional because they define *how well* the system performs those functions (e.g., how fast, how securely, or how reliably the payment is processed).

2. What is the difference between functional requirements and quality factors?

- **Functional Requirements:** Specify the exact behavior, inputs, and outputs of a system (e.g., "The system must capture a photo").
- **Quality Factors:** Specify the constraints and performance standards of the system (e.g., "The photo capture process must load in under 1 second and handle 1,000 concurrent users").

3. List the three categories of McCall's Software Quality Factors.

- **Product Operation:** How well the software runs.
- **Product Revision:** How easily the software can be changed.
- **Product Transition:** How easily the software can be moved or integrated.

4. Define Correctness with an example.

Correctness is the extent to which a program fulfills its specifications and meets the client's objectives.

- *Example:* If an E-KYC(Electronic-Know Your Customer) onboarding system is specified to perfectly extract text from an uploaded National ID card using OCR(Optical Character Recognition), achieving accurate extraction without data loss demonstrates correctness.

5. What is meant by Reliability? Give one measurable reliability requirement.

Reliability refers to the software's ability to provide continuous service without failure over a specified period.

- *Measurable Example:* "The backend API must maintain a 99.9% uptime, resulting in no more than 43 minutes of downtime per month."

6. Explain the role of Integrity in software quality.

Integrity dictates the system's security and its ability to prevent unauthorized access to data. In an enterprise system, high integrity ensures that sensitive customer information remains encrypted and safe from breaches.

7. What is the difference between Maintainability and Flexibility?

- **Maintainability:** The effort required to locate and fix a *bug* or error in the software.
- **Flexibility:** The effort required to *modify* or add new features to the software to adapt to changing business needs.

8. Why is Testability important during software development?

Testability ensures that developers can easily verify if a module works correctly or if a bug has been successfully resolved. Software designed with good logging and diagnostic tools saves significant time during debugging and QA cycles.

9. Define Portability with a practical example.

Portability is the ease with which software can be transferred from one hardware or software environment to another.

- *Example:* Successfully migrating a Java Spring Boot backend database from an on-premise local server to an AWS cloud environment without having to rewrite the core application logic.

10. What is Interoperability and why is it important in enterprise software?

Interoperability is the ability of a system to connect and exchange data smoothly with other distinct systems. It is vital in enterprise environments because modern applications rarely operate in isolation (e.g., a geolocation-based attendance application must seamlessly push data into a separate, centralized HRMS API).

Part B: Analytical Questions (5 marks each)

1. Compare the three categories of McCall's Software Quality Factors using suitable examples.

- **Product Operation (Running):** Focuses on the immediate user experience and system stability. *Example:* Ensuring a banking portal's liveness-check algorithm executes quickly (Efficiency) and securely (Integrity).
- **Product Revision (Adapting):** Focuses on developer interaction with the codebase over time. *Example:* Structuring the code cleanly so that adding a new "Passport Verification" module takes minimal effort (Flexibility).
- **Product Transition (Moving):** Focuses on the system's adaptability to new infrastructure. *Example:* Reusing a previously built OCR component in a completely new loan-processing application (Reusability).

2. Explain how poor software requirements can negatively affect software quality.

If non-functional requirements are ignored during the initial Software Requirements Specification (SRS), the resulting architecture will be flawed. For instance, if data load (Efficiency) is not specified, developers might build a system that works perfectly for 10 users

but crashes for 10,000. This leads to massive technical debt, severe operational downtime, and costly code rewrites.

3. A hospital information system requires 99.999% availability. Which McCall quality factors are most critical? Justify your answer.

- **Reliability:** The system must not crash, as downtime could literally risk patient lives.
- **Correctness:** Medical data must be precisely accurate; incorrect dosages or patient records are catastrophic.
- **Integrity:** Patient health records are highly sensitive and require strict privacy controls to prevent unauthorized access.

4. Distinguish between Reliability, Safety, and Survivability.

- **Reliability:** Focuses on the *frequency* of failure (e.g., Mean Time Between Failures).
- **Safety (Deutsch & Willis):** Focuses on preventing the system from causing hazards or damage when it does fail (e.g., triggering a fallback alarm).
- **Survivability (Deutsch & Willis):** Focuses on business continuity and the speed of recovery after a failure occurs (e.g., Mean Time To Recover).

5. Explain why Reusability can reduce software development cost and time. By designing software in modular, decoupled blocks (like microservices), components can be dropped into new projects without starting from scratch. Reusing a heavily tested, production-ready module eliminates the need to spend billable hours on redesigning, recoding, and re-testing that specific functionality.

Part C: Scenario-Based Questions (8–10 marks each)

Answer to Scenario 1 & 2: Banking Application

Based on McCall's Software Quality Model, the relevant quality factors are:

1. Prevent unauthorized access → Integrity

- **Why it is important:** It ensures the system uses robust security (like encryption and authentication) to protect sensitive user data, prevent cyberattacks, and stop financial fraud.

2. Process transactions within 2 seconds → Efficiency

- **Why it is important:** It ensures the software optimally utilizes hardware resources (CPU, memory, database) so that transactions are processed instantly without lagging or timing out during heavy traffic.

3. Operate continuously with minimal downtime → Reliability

- **Why it is important:** Banking requires 24/7 availability. High reliability ensures the system is fault-tolerant and does not crash, guaranteeing that customers always have uninterrupted access to their funds.

3. E-commerce Company Scenario *Context:* Run on Windows, Linux, Android, Cloud, and integrate with payment gateways.

- **Portability (Highest Priority):** The application must operate seamlessly across entirely different operating systems and hardware architectures (mobile, desktop, and cloud).
- **Interoperability (Highest Priority):** The platform cannot process sales without talking to external, third-party payment gateways (like Stripe or PayPal). The APIs must integrate flawlessly.
- *Justification:* Without Portability, the multi-platform strategy fails. Without Interoperability, the core business function (collecting money) fails.

4. Modifying Products for Different Clients

- **Flexibility:** The most critical factor. The codebase must be designed so that custom features can be toggled or added for Client A without breaking the core product for Client B.
- **Maintainability:** Frequent modifications increase the risk of bugs. The code must be clean and well-documented so that developers can quickly trace and resolve client-specific errors.

5. McCall's vs. Alternative Models McCall's foundational model lacks considerations for modern operational tracking and scaling.

- **Evans & Marciniak** added *Verifiability* (ease of verifying design/code structure) and *Expandability* (scalability).
- **Deutsch & Willis** added *Safety*, *Manageability* (version control), and *Survivability*.
- *Most Valuable Today: Expandability (Scalability).* In the era of cloud computing and microservices, an application's ability to dynamically scale hardware resources up or down based on traffic spikes is a mandatory requirement for enterprise software.

Part D: Higher-Order Thinking (10–15 marks each)

1. Is McCall's Model still relevant for AI-based and Cloud-native systems? Yes, it acts as a strong foundation, but it requires modernization.

- *Relevance:* Core factors like Reliability, Integrity, and Interoperability are just as vital today. An AI system must be reliable, and a cloud system must interoperate with various APIs.
- *Limitations:* McCall does not account for cloud-specific traits like *Elasticity* (auto-scaling). Furthermore, for AI systems, *Correctness* becomes ambiguous. AI models deal with probabilities rather than binary right/wrong outputs. Therefore, modern models must include factors for "Model Drift" and "Data Privacy."

2. LMS Quality Requirements Specification (Using McCall)

- **Operation:**
 - *Usability*: Intuitive UI using accessible front-end templates so non-technical students can navigate courses instantly.
 - *Efficiency*: Video playback must stream without buffering on standard broadband.
- **Revision:**
 - *Testability*: Automated CI/CD pipelines must verify that new quiz features do not break existing gradebook calculations.
- **Transition:**
 - *Interoperability*: Must sync grades directly to the university's central administrative database and integrate with external SSO (Single Sign-On).

3. Influence on SRS with Examples Quality factors transform an SRS from a simple "wishlist" into an actionable architectural blueprint. If a client asks for a user dashboard (Functional), the SRS must specify the *Efficiency* and *Reliability* (Non-functional).

- *Example*: If the SRS defines high *Flexibility* and *Reusability*, the software team will likely choose a Microservices architecture using Docker and Node.js rather than a monolithic architecture. The quality factor directly dictates the technical stack and design pattern.

4. Autonomous Vehicle Control System Priorities

- **Safety (Deutsch & Willis)**: The absolute highest priority. If sensors fail, the system must fail safely (e.g., slowly pulling the car over) to prevent loss of life.
- **Reliability (McCall)**: The software cannot crash while driving at 70 mph; the MTBF (Mean Time Between Failures) must be practically infinite.
- **Efficiency (McCall)**: Real-time data processing is required. The system must process camera and LIDAR data in milliseconds to apply brakes in time.

Viva / Discussion Questions

- **Which software quality factor is the most difficult to measure? Why?**
 - *Usability* and *Maintainability*. They are highly subjective and depend on human factors (e.g., how intuitive a user finds an interface or how easily a developer understands someone else's code).
- **Can software be highly efficient but have poor usability? Explain.**
 - Yes. A backend system might process complex algorithms in milliseconds (high efficiency) but output the results via a command-line interface that requires complex syntax to operate (poor usability).
- **Why should clients be involved in defining software quality requirements?**
 - Clients understand the business context. They know what degree of downtime will cost them money or what level of security their industry regulators demand.
- **Are quality factors equally important for every software project? Justify your answer.**

- No. A mobile game prioritizes Usability and Efficiency (graphics rendering), whereas a banking portal prioritizes Integrity (security) and Reliability over flashy UI.
- **Which quality factor is most critical for AI-powered healthcare systems?**
 - *Safety* and *Correctness*. An incorrect AI diagnosis or a system failure during robotic surgery directly threatens human life.